

CADERNO DE ESTUDOS

SQL

Structured Query Language · Banco de Dados

Organizado por temas

DDL · DML · DQL · DCL · DTL

Relacionamentos & Modelos de Dados

Última atualização: 26/06/2026 | Diego | uso no GoodNotes

Como usar este caderno

Este é o seu caderno único de SQL, **organizado por temas** e não por ordem de aula. Cada novo conteúdo é encaixado no tema a que pertence, mantendo o mesmo padrão visual. As cores indicam o **tipo** de conteúdo, para você localizar a informação rapidamente. Ao lado de cada título há uma **data discreta** indicando quando aquele tema foi estudado.

CONCEITO

Definição ou ideia central.

CÓDIGO SQL

Sintaxe e exemplos prontos.

ATRIBUTO / REGRA

Restrições e propriedades de colunas.

DICA / ATENÇÃO

Pontos que costumam confundir.

MAPA MENTAL

Visão geral em diagrama.

NA PRÁTICA

Resumo e o que fazer com isso.

Índice por temas

01	O que é Banco de Dados	Conceito
02	O que é SQL	Conceito
03	Padronização (ANSI / ISO)	Conceito
04	As 5 sublinguagens do SQL	Mapa mental
05	DDL — Definição (CREATE, ALTER, DROP)	Código
06	DML — Manipulação (INSERT, UPDATE, DELETE)	Código
07	DQL — Consulta (SELECT)	Código
08	DCL — Controle (GRANT, REVOKE)	Código
09	DTL — Transação (COMMIT, ROLLBACK)	Código
10	Atributos de colunas	Atributo
11	Chaves: PK, FK, UNIQUE e Surrogada	Chaves
12	Relacionamentos: 1:1, 1:N, N:M	Diagrama

13	Modelos de Dados	Mapa mental
14	Filtros e a cláusula WHERE	Consultas avançadas
15	Funções de agregação e GROUP BY	Consultas avançadas
16	ORDER BY e operações aritméticas	Consultas avançadas
17	Tipos de dados (datatypes)	Tipos e funções
18	Funções de string (texto)	Tipos e funções
19	Funções numéricas e de data	Tipos e funções
20	Conversão e condicionais (CASE, COALESCE)	Tipos e funções
21	JOINS — juntando tabelas	Juntando tabelas
22	UNION — empilhando resultados	Juntando tabelas
23	Subqueries (subconsultas)	Consultas avançadas
24	Views (visões)	Consultas avançadas

01 O que é Banco de Dados

estudado em 26 jun 2026

CONCEITO

Um **banco de dados** é uma coleção organizada de dados armazenados eletronicamente em um sistema de computador. Ele permite **gerenciar, buscar e manipular** essas informações de forma eficiente.

Tabelas: a estrutura básica

Dentro de um banco, os dados ficam em **tabelas**, organizadas em **linhas** e **colunas**:

- **Linha (registro):** representa uma ocorrência completa — por exemplo, um cliente específico.
- **Coluna (campo / atributo):** representa uma característica do dado — por exemplo, "nome" ou "CPF".

ANALOGIA

Pense numa planilha do Excel. Cada **aba** é uma tabela, cada **linha** é um cadastro e cada **coluna** é uma informação daquele cadastro. Um banco de dados é isso, porém preparado para muitos dados e muitos acessos ao mesmo tempo, com regras de integridade.

NA PRÁTICA

Em outras palavras: banco de dados = onde a informação mora, de forma organizada.

O que isso significa: toda consulta SQL que você escrever depois vai agir sobre essas linhas e colunas.

02 O que é SQL

estudado em 26 jun 2026

CONCEITO

SQL (Structured Query Language) é a "Linguagem Estruturada de Consulta". É o idioma que usamos para conversar com o banco de dados: criar tabelas, inserir, alterar e consultar dados.

- Criada no **início dos anos 70**, pela **IBM**.
- Linguagem de **altíssimo nível**: bem próxima da linguagem humana (inglês). Comandos como `SELECT`, `INSERT` e `DELETE` já dizem o que fazem.
- É uma linguagem **declarativa**: você descreve *o resultado que quer*, não o passo a passo de como obtê-lo.
- Ao longo do tempo surgiram novos **dialetos**, derivando e evoluindo o SQL.

DICA – DECLARATIVO VS. IMPERATIVO

Declarativo (SQL): "quero os clientes de SP". Você não diz como percorrer a tabela; o banco resolve.

Imperativo (ex.: um `for` em Python): você escreve cada passo da busca.

Por que importa: isso explica por que SQL é tão enxuto. Você foca no "o quê", e o motor do banco cuida do "como".

DICA – O QUE É "DIALETO"

Um **dialeto** é uma versão do SQL adaptada por um fabricante. Exemplos: **MySQL**, **PostgreSQL**, **SQL Server (T-SQL)**, **Oracle (PL/SQL)**. O núcleo é o mesmo; mudam alguns detalhes de sintaxe.

Para que o SQL funcionasse de forma parecida em qualquer sistema, ele foi padronizado por dois órgãos:

Órgão	O que é	Ano
ANSI	American National Standards Institute — instituto de padrões dos EUA.	1986
ISO	International Organization for Standardization — padrões internacionais.	1987

CONCEITO

O SQL é a linguagem usada em praticamente **todos os SGBDs** do mercado.

DICA – O QUE É SGBD

SGBD = Sistema de Gerenciamento de Banco de Dados (em inglês, DBMS). É o software que administra o banco: guarda os dados, garante segurança e executa seus comandos SQL. Exemplos: MySQL, PostgreSQL, Oracle, SQL Server.

ATENÇÃO

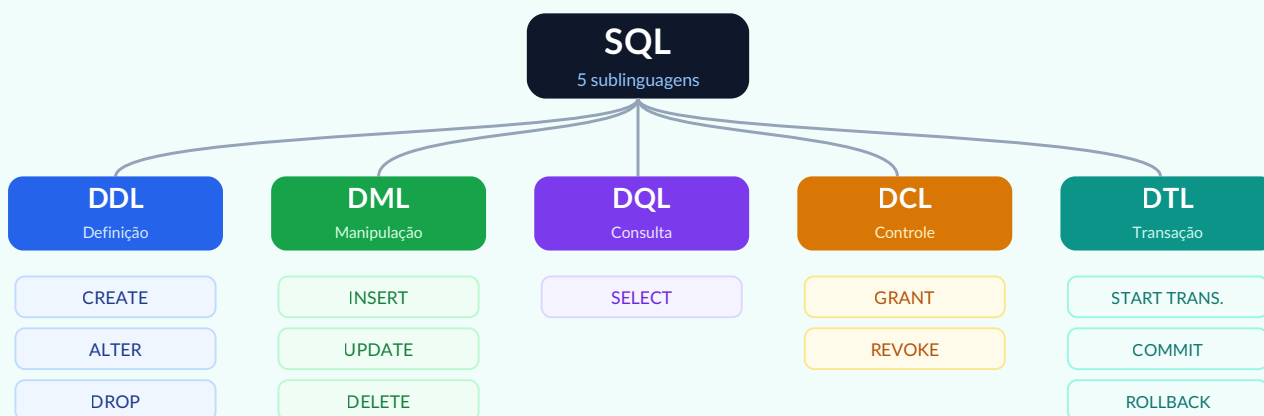
Mesmo com a padronização, **cada fabricante tem suas características** próprias. Um comando pode funcionar no MySQL e exigir pequeno ajuste no SQL Server. Por isso, sempre confira a documentação do banco que você está usando.

04 As 5 sublinguagens do SQL

estudado em 26 jun 2026

O SQL não é um bloco único. Ele é formado por **cinco conjuntos de comandos**, cada um com uma finalidade. Entender essa divisão organiza todo o resto da matéria.

Mapa mental – o "guarda-chuva" SQL



Defino a estrutura (DDL) → coloco/mudo dados (DML) → consulto (DQL)
→ controlo quem acessa (DCL) → garanto a transação (DTL)

Sigla	Nome	Para quê serve
DDL	Data Definition Language	Definir a estrutura (criar/alterar/excluir tabelas).
DML	Data Manipulation Language	Manipular os dados (inserir, alterar, deletar registros).
DQL	Data Query Language	Consultar dados (SELECT).
DCL	Data Control Language	Controlar acessos e permissões.
DTL	Data Transaction Language	Gerenciar transações (confirmar ou desfazer).

NA PRÁTICA

Quando estudar qualquer comando novo, primeiro pergunte: **"de qual sublinguagem ele faz parte?"**. Isso já diz o propósito dele e evita confusão.

05 DDL – Linguagem de Definição de Dados

estudado em 26 jun 2026

CONCEITO

A **DDL** cuida da **estrutura** do banco. Ela não mexe nos dados em si, e sim no "esqueleto": as tabelas e suas colunas.

Comando	Ação
CREATE	Cria uma estrutura (ex.: uma tabela).
ALTER	Altera uma estrutura existente.
DROP	Exclui uma estrutura.

CREATE TABLE – criar uma tabela

CÓDIGO SQL

```
-- Forma geral:
CREATE TABLE nomeTabela (campo tipo atributo, ... );

-- Exemplo: tabela de clientes
CREATE TABLE CLIENTE (
  codigo      int      NOT NULL PRIMARY KEY,
  nome        varchar(30) NOT NULL,
  cpf         int      UNIQUE,
  uf          char(2)   DEFAULT null,
  nascimento  date     DEFAULT '0000-00-00',
  telefone    char(8)   DEFAULT null,
  sexo        char(1)
);
```

LENDO A SINTAXE

Cada coluna segue o padrão **nome • tipo • atributos**. Exemplo: `nome varchar(30) NOT NULL` = coluna "nome", texto de até 30 caracteres, que não pode ficar vazia.

ALTER TABLE — alterar uma tabela

CÓDIGO SQL

```
-- Incluir uma nova coluna:  
ALTER TABLE CLIENTE ADD email VARCHAR(20);  
ALTER TABLE CLIENTE ADD fax VARCHAR(18) AFTER uf;  
  
-- Eliminar uma coluna existente:  
ALTER TABLE CLIENTE DROP fax;  
  
-- Alterar o tipo de dado de uma coluna:  
ALTER TABLE CLIENTE MODIFY email VARCHAR(40);
```

DROP TABLE — excluir uma tabela

CÓDIGO SQL

```
DROP TABLE [nome da tabela];
```

ATENÇÃO — CUIDADO COM DROP E ALTER...DROP

DROP TABLE apaga a tabela inteira, com todos os dados, e não tem "lixeira". **ALTER TABLE ... DROP coluna** apaga só a coluna. São ações destrutivas: confirme o ambiente antes de rodar.

06 DML – Linguagem de Manipulação de Dados

estudado em 26 jun 2026

CONCEITO

A **DML** trabalha com o **conteúdo** das tabelas: inserir, alterar e apagar **registros** (linhas). A estrutura já existe; aqui mexemos nos dados.

Comando	Ação
INSERT	Inserir dados (novas linhas).
UPDATE	Alterar dados existentes.
DELETE	Deletar dados.

INSERT INTO – inserir registros

CÓDIGO SQL

```
INSERT INTO TABELA VALUES (valores);

-- Exemplo:
INSERT INTO TABELA VALUES
(123, 'Allan Rucker', '12345', 'SP',
 '1234-12-12', '123456', 'M', 'allan@email.com');
```

UPDATE – atualizar registros

CÓDIGO SQL

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

DELETE – apagar registros

CÓDIGO SQL

```
DELETE FROM table_name WHERE condition;
```

ATENÇÃO – O PERIGO DE ESQUECER O WHERE

Em **UPDATE** e **DELETE**, a cláusula **WHERE** define *quais* linhas serão afetadas. **Sem WHERE**, o comando atinge **TODAS** as linhas da tabela. Um `DELETE FROM CLIENTE;` sem **WHERE** apaga todos os clientes.

NA PRÁTICA

Hábito de segurança: antes de rodar um UPDATE/DELETE, escreva primeiro um `SELECT` com o mesmo `WHERE` para ver exatamente quais linhas serão afetadas.

07 DQL – Linguagem de Consulta de Dados

estudado em 26 jun 2026

CONCEITO

A **DQL** é dedicada a **consultar** dados. O comando central é o **SELECT**, o mais usado no dia a dia de quem trabalha com dados.

O **SELECT** também permite:

- **Ordenação** de dados (ex.: por nome, por data).
- **Agrupamento** de dados (juntar por categoria).
- **Funções aritméticas** (somar, contar, calcular médias).
- **Filtros** de seleção (escolher só o que interessa).

SELECT – consultar

CÓDIGO SQL

```
SELECT *  
FROM TABELA  
WHERE nome = 'Allan';
```

LENDO A CONSULTA

`SELECT *` = "traga todas as colunas". `FROM TABELA` = "da tabela TABELA". `WHERE nome = 'Allan'` = "apenas as linhas onde a coluna nome for igual a Allan".

Dica de leitura: mentalmente, o banco lê primeiro o `FROM`, depois o `WHERE` e por último o `SELECT`.

NA PRÁTICA

Como você está em transição para Dados, o `SELECT` será sua ferramenta mais frequente. Vale dominar bem `WHERE`, `ORDER BY` e `GROUP BY` nas próximas aulas.

08 DCL – Linguagem de Controle de Dados

estudado em 26 jun 2026

CONCEITO

A DCL controla quem pode fazer o quê no banco. É a parte de **permissões e segurança** de acesso.

Comando	Ação
GRANT	Habilita (concede) acesso a dados e operações.
REVOKE	Revoga (retira) acesso a dados e operações.

ANALOGIA – CHAVES DE ACESSO

Pense num prédio. **GRANT** é entregar a chave de uma sala a alguém. **REVOKE** é recolher essa chave. O DCL define quem entra em quais "salas" (tabelas) e o que pode fazer lá dentro.

NA PRÁTICA

No seu contexto de serviços financeiros, DCL é o que garante que cada pessoa veja apenas os dados que tem permissão. É um pilar de **segurança e governança** de dados.

09 DTL – Linguagem de Transação de Dados

estudado em 26 jun 2026

CONCEITO

A DTL gerencia **transações**. Uma transação é um conjunto de operações tratado como uma **unidade única**: ou tudo dá certo e é confirmado, ou nada é aplicado.

Comando	Ação
START TRANSACTION	Inicia a transação.
COMMIT	Concretiza (confirma) a transação.
ROLLBACK	Anula (desfaz) a transação.

ANALOGIA – TRANSFERÊNCIA BANCÁRIA

Numa transferência, dois passos precisam acontecer juntos: **debitar** de uma conta e **creditar** em outra. Se o sistema cair no meio, não pode debitar sem creditar.

COMMIT = confirma os dois passos juntos. **ROLLBACK** = se algo falhou, desfaz tudo e volta ao estado inicial. Ninguém perde dinheiro.

NA PRÁTICA

Em outras palavras: transação garante o "tudo ou nada". **Por que importa:** é o que dá **confiabilidade** a sistemas financeiros, evitando estados pela metade.

10 Atributos de colunas

estudado em 26 jun 2026

CONCEITO

Atributos são regras e propriedades que você aplica a uma coluna no momento de criá-la. Eles controlam o que aquela coluna aceita ou como se comporta.

Atributo	O que faz
NULL / NOT NULL	Permite ou não valores nulos na coluna. NULL é diferente de vazio : nulo = "ausência de valor / desconhecido".
SIGNED / UNSIGNED	Permite ou não números negativos. UNSIGNED = só positivos (útil para idade, quantidade).
AUTO_INCREMENT	Gera sequências automáticas (contadores). Cada novo registro recebe o próximo número.
ZEROFILL	Preenche o número completando com zeros à esquerda (ex.: 7 vira 007).

DICA – NULL NÃO É ZERO NEM ESPAÇO

NULL significa "não há informação". É diferente de **0** (um número) e de **"** (texto vazio). Exemplo: um cliente sem telefone cadastrado tem telefone **NULL**, não telefone **"0"**.

NA PRÁTICA

AUTO_INCREMENT conecta com o próximo tópico: é a forma mais comum de gerar uma **chave primária** automática (a chave surrogada).

11 Chaves: PK, FK, UNIQUE e Surrogada

estudado em 26 jun 2026

CONCEITO – O QUE SÃO CHAVES

Chaves são atributos (ou conjuntos de atributos) que identificam de forma única um registro numa tabela. São essenciais para a **integridade dos dados**: previnem duplicatas e permitem criar **relacionamentos** entre tabelas. Sem chaves, seria impossível gerenciar dados de forma eficaz.

CHAVE PRIMÁRIA (PRIMARY KEY – PK)

Atributo (ou conjunto) que identifica de forma única **cada registro** da tabela. Garante que não existam dois registros com a mesma PK. Só pode haver **uma por tabela** e é sempre **NOT NULL**.

Exemplo: na tabela `Clientes`, o `ID` do `cliente` é a PK — cada cliente tem um identificador único.

CHAVE ESTRANGEIRA (FOREIGN KEY – FK)

Campo de uma tabela que cria um **vínculo** com a PK de outra tabela. Mantém a **integridade referencial**: o valor da FK precisa sempre corresponder a uma PK válida na tabela relacionada.

Exemplo: em `Pedidos`, o campo `id_cliente` é FK e aponta para a PK de `Clientes`. Não existe pedido de um cliente que não existe.

PK E FK TRABALHANDO JUNTAS

A **PK** identifica a linha dentro da própria tabela. A **FK** é a referência dessa PK em outra tabela, criando a ligação. É esse par que torna possível relacionar tabelas (próximo tema).

CONCEITO – RESTRIÇÕES UNIQUE E PRIMARY KEY

Ambas **asseguram que os valores de uma ou mais colunas sejam únicos** em cada linha da tabela. Servem para não ter duplicidade (ex.: dois clientes com o mesmo CPF).

Restrição	Característica
PRIMARY KEY	Identificador principal da linha. Só pode haver uma por tabela. É única e obrigatoriamente NOT NULL .
UNIQUE	Garante unicidade, mas pode haver várias na mesma tabela. Usada em colunas que também devem ser únicas (ex.: e-mail, CPF).

REGRA – RELAÇÃO COM NOT NULL

Colunas definidas como **PRIMARY KEY** ou **UNIQUE** geralmente também precisam ser **NOT NULL**. O atributo **NOT NULL** impede valores nulos e é **obrigatório** em colunas com restrição PRIMARY KEY.

CONCEITO – CHAVE SURROGADA (SURROGATE KEY)

Uma **chave surrogada** é uma chave **artificial**, geralmente numérica, criada apenas para servir de chave primária. Ela substitui uma **chave natural** que seria complexa ou não ideal.

DICA – SURROGADA VS. NATURAL

Chave natural: um dado real do mundo que identifica a linha, como o CPF.

Chave surrogada: um número sem significado de negócio, criado só para identificar (ex.: `codigo` com `AUTO_INCREMENT`).

Por que usar surrogada: é simples, estável e rápida. Se a regra de negócio mudar (ex.: passar a aceitar clientes estrangeiros sem CPF), a chave artificial continua funcionando.

12 Relacionamentos entre tabelas

estudado em 26 jun 2026

CONCEITO

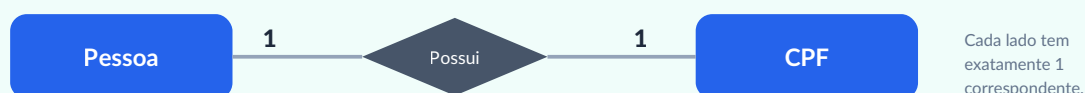
Relacionamento é a **ligação** entre tabelas, feita por chaves: a **Primary Key (PK)** de uma tabela aparece em outra como **Foreign Key (FK)**. A "cardinalidade" diz **quantas** linhas de um lado se ligam ao outro.

DICA – O QUE É FOREIGN KEY (FK)

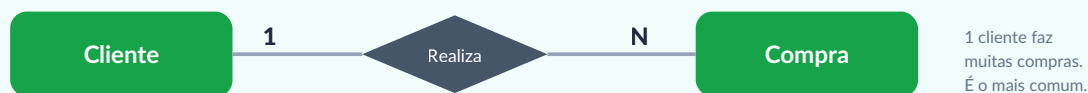
A **chave estrangeira** é uma coluna que aponta para a chave primária de outra tabela. É o que conecta, por exemplo, uma **Compra** ao **Cliente** que a realizou. Ela garante a **integridade referencial** (não existe compra de um cliente que não existe).

Diagrama – os três tipos de cardinalidade

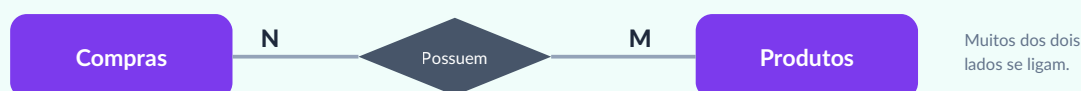
Relacionamento 1:1



Relacionamento 1:N



Relacionamento N:M



Como ler:

- Retângulo = entidade (tabela)
- Losango = o relacionamento (ação)
- 1, N, M = cardinalidade (quantos de cada lado participam)
- N:M geralmente vira uma tabela intermediária na implementação real.

NA PRÁTICA

1:1 → cada um, um só (**Pessoa** ↔ **CPF**). **1:N** → um para muitos (**Cliente** → **Compras**). **N:M** → muitos para muitos (**Compras** ↔ **Produtos**).

Atenção: bancos relacionais não implementam N:M diretamente. Cria-se uma **tabela intermediária** (associativa) que transforma o N:M em dois relacionamentos 1:N.

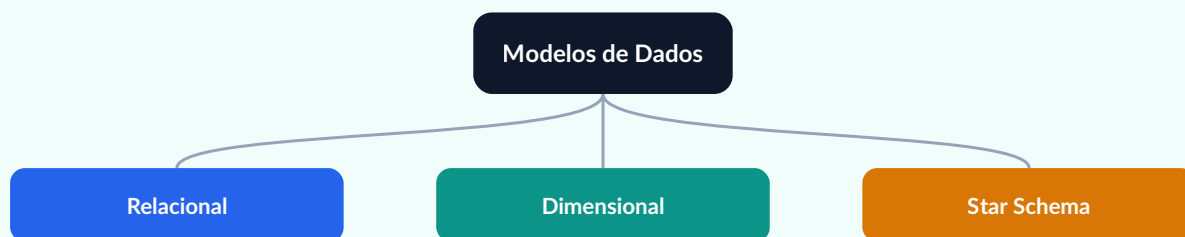
13 Modelos de Dados

estudado em 26 jun 2026

CONCEITO

Modelos de dados são formas de organizar e estruturar os dados num banco. A escolha depende do objetivo: operação do dia a dia ou análise.

Mapa mental – quando usar cada modelo



Detalhamento

MODELO RELACIONAL

Organiza dados em **tabelas inter-relacionadas** por chaves primárias e estrangeiras. Ideal para **operações transacionais**: inserir, atualizar, excluir e consultar com **garantia de integridade referencial**. É o modelo da maior parte dos sistemas do dia a dia (ex.: o cadastro de clientes do banco).

MODELO DIMENSIONAL

Usado principalmente em **data warehouses** e sistemas de **BI** (Business Intelligence). Organiza os dados em duas categorias: **fatos** (dados quantitativos, como vendas) e **dimensões** (contexto/ descritores, como tempo, produto ou localização).

STAR SCHEMA (ESQUEMA ESTRELA)

É um tipo de modelo dimensional: uma **tabela de fatos central** conectada a várias **tabelas de dimensão**, formando o desenho de uma estrela. É simples e eficiente para consultas, facilitando **análises rápidas** e agregações de dados.

NA PRÁTICA – CONEXÃO COM SUA TRANSIÇÃO PARA DADOS

Relacional = mundo da **operação** (registrar o que acontece). **Dimensional / Star Schema** = mundo da **análise** (entender o que aconteceu). Na sua migração para Dados e BI, você sairá do relacional em direção ao dimensional.

14 Filtros e a cláusula WHERE

estudado em 26 jun 2026

CONCEITO – O QUE É O WHERE

O **WHERE** ("onde") filtra os resultados de uma consulta: define **quais linhas** aparecem. O banco testa a condição **linha por linha** – se for verdadeira, a linha aparece; se falsa, é ignorada.

Por que usar: foco (ver só o que importa) e **performance** (processa menos dados, busca mais rápida).

Tabela de exemplo – PESSOAS

ID	NOME	IDADE	FILHOS
1	Eliana	20	0
2	Miguel	39	3
3	Raquel	36	2
4	João	22	1
5	Marta	48	2

Operadores de comparação

O QUE SÃO

Fazem uma **pergunta** aos dados e devolvem um valor **booleano** (verdadeiro ou falso). Booleano = tipo de dado com apenas dois estados possíveis.

Operador	Significado
=	Igual a
!= ou <>	Diferente de
>	Maior que
<	Menor que
>=	Maior ou igual
<=	Menor ou igual

CÓDIGO SQL

```
-- Idade maior ou igual a 39:
SELECT * FROM PESSOAS WHERE IDADE >= 39;
-- Resultado: Miguel (39) e Marta (48)

-- Diferente de zero filhos:
SELECT * FROM PESSOAS WHERE FILHOS <> 0;
-- Resultado: Miguel, Raquel, João, Marta
```

Álgebra booleana: AND, OR, NOT

COMBINANDO CONDIÇÕES

AND (E): só retorna se **todas** as condições forem verdadeiras. Analogia: receita que exige "ovo E leite" — falta um, não dá.

OR (OU): retorna se **pelo menos uma** for verdadeira. Analogia: "suco OU refrigerante".

NOT (NÃO): inverte — traz o oposto da condição.

CÓDIGO SQL

```
SELECT * FROM PESSOAS
WHERE IDADE IN (39, 48)
      AND FILHOS > 2;
-- Só Miguel: tem 39 anos E mais de 2 filhos (3).
-- Marta sai: tem 48, mas só 2 filhos.
```

Intervalo, lista, texto e nulos

Operador	O que faz
BETWEEN	Valor dentro de um intervalo (inclui as pontas). Ex.: IDADE BETWEEN 20 AND 23.
IN	Valor que coincide com algum item de uma lista. Ex.: UF IN ('SP','RJ','MG').
LIKE	Busca padrões em texto. % = vários caracteres; _ = um caractere.
IS NULL / IS NOT NULL	Campo vazio (sem informação) ou preenchido.
DISTINCT	Remove duplicatas; retorna só valores exclusivos.

CÓDIGO SQL

```
-- Nomes que começam com M:
SELECT * FROM PESSOAS WHERE NOME LIKE 'M%';
-- Miguel e Marta

-- Idade entre 20 e 23 (inclui as pontas):
SELECT * FROM PESSOAS WHERE IDADE BETWEEN 20 AND 23;
-- Eliana (20) e João (22)
```

ATENÇÃO – LIKE E WILDCARDS

`%` representa zero ou muitos caracteres; `_` representa exatamente um. `'M%'` = começa com M.
`'%a'` = termina com a. `'_a%'` = a segunda letra é a.

NA PRÁTICA

O `WHERE` é o coração da consulta no dia a dia de dados. Combine operadores de comparação com `AND/OR/NOT` e refine com `LIKE`, `BETWEEN`, `IN` e `IS NULL`. **Próxima ação:** teste cada exemplo mentalmente na tabela `PESSOAS`.

15 Funções de agregação e GROUP BY

estudado em 26 jun 2026

CONCEITO

Funções de agregação calculam um **único resultado** a partir de um **conjunto de valores** (várias linhas). Em vez de ver cada linha, você vê um resumo: total, média, maior, menor.

Função	O que faz
COUNT	Conta linhas. COUNT(*) conta todas; COUNT(DISTINCT col) conta só valores diferentes. Ignora NULL.
SUM	Soma os valores de uma coluna.
AVG	Calcula a média aritmética.
MIN	Menor valor.
MAX	Maior valor.

CÓDIGO SQL

```
-- Resumo da tabela PESSOAS:
SELECT
  COUNT(*)      AS total,
  AVG(IDADE)    AS media_idade,
  MAX(FILHOS)   AS max_filhos
FROM PESSOAS;
-- total=5, media_idade=33, max_filhos=3
```

AS (ALIAS)

AS dá um **nome temporário** a uma coluna ou tabela, deixando o resultado mais legível. COUNT(*) AS total faz a coluna aparecer como "total".

GROUP BY – AGRUPAR

Agrupa as linhas por uma coluna para aplicar a agregação **a cada grupo separadamente**. Sem GROUP BY, a agregação trata a tabela inteira como um só grupo.

CÓDIGO SQL

```
-- Quantas pessoas têm cada quantidade de filhos:
SELECT FILHOS, COUNT(*) AS qtd
FROM PESSOAS
GROUP BY FILHOS;
-- 0 tem 1, 1 tem 1, 2 tem 2, 3 tem 1
```

ATENÇÃO – COUNT E NULL

`COUNT(*)` conta todas as linhas. `COUNT(coluna)` conta só as linhas em que a coluna **não é NULL**. Útil para saber quantos registros têm aquele dado preenchido.

NA PRÁTICA

Agregação + GROUP BY é a base de qualquer relatório: "total por categoria", "média por mês", "contagem por status". **Próxima ação:** pense numa pergunta do seu trabalho que comece com "quantos" ou "qual a média de".

16 ORDER BY e operações aritméticas

estudado em 26 jun 2026

ORDER BY – ORDENAR

Define a **ordem** dos resultados. **ASC** = crescente (A-Z, 0-9), é o padrão. **DESC** = decrescente. É a **última cláusula** do SELECT. Pode ordenar até por colunas não selecionadas e pela **posição numérica** da coluna.

CÓDIGO SQL

```
-- Do mais velho para o mais novo:
SELECT NOME, IDADE
FROM PESSOAS
ORDER BY IDADE DESC;
-- Marta(48), Miguel(39), Raquel(36), João(22), Eliana(20)
```

OPERAÇÕES ARITMÉTICAS

Dá para calcular direto no SELECT com + (adição), - (subtração), * (multiplicação), / (divisão) e % (módulo = resto da divisão).

ATENÇÃO – DIVISÃO POR ZERO

Dividir por zero **gera erro**. Use o `WHERE` para descartar esses casos antes do cálculo.

CÓDIGO SQL

```
-- Evita divisão por zero filtrando FILHOS > 0:
SELECT NOME, IDADE / FILHOS AS idade_por_filho
FROM PESSOAS
WHERE FILHOS > 0;
-- Eliana (0 filhos) é desconsiderada
```

ORDEM DE EXECUÇÃO (PRECEDÊNCIA)

Para o cálculo sair certo, o SQL segue esta hierarquia: 1. Parênteses () ; 2. Expoentes; 3. Multiplicação * e divisão / ; 4. Adição + e subtração -. Use parênteses para deixar a intenção explícita.

NA PRÁTICA

`ORDER BY` organiza a leitura; as operações aritméticas geram métricas direto na consulta. **Próxima ação:** filtre com `WHERE` antes de dividir, para nunca cair em divisão por zero.

17 Tipos de dados (datatypes)

estudado em 26 jun 2026

CONCEITO

Tipos de dados (datatypes) são categorias que definem o tipo de informação que uma coluna pode armazenar. Especificam o **formato**, o **tamanho** e a **natureza** do dado, garantindo armazenamento consistente e eficiente.

Numéricos

Tipo	Tamanho	Faixa / uso
BOOLEAN	1 bit	0 ou 1 (FALSE/TRUE).
TINYINT	1 byte	-128 a 127.
SMALLINT	2 bytes	-32.768 a 32.767 (0 a 65.535 se UNSIGNED).
INT	4 bytes	cerca de -2,1 bi a +2,1 bi.
BIGINT	8 bytes	números muito grandes.
FLOAT	4 bytes	decimais (ponto flutuante).
DOUBLE	8 bytes	decimais com mais precisão.

Texto (string)

Tipo	Tamanho	Uso
CHAR	fixo	Texto de tamanho fixo. Ex.: uf CHAR(2).
VARCHAR	variável (+1 byte)	Texto de tamanho variável. Ex.: nome VARCHAR(30).

Data e hora

Tipo	Tamanho	Guarda
DATE	3 bytes	Só a data (ano, mês, dia).
TIME	3 bytes	Só a hora.
DATETIME	8 bytes	Data e hora juntas.
TIMESTAMP	4 bytes	Um ponto específico no tempo.

CHAR VS VARCHAR

CHAR é fixo: sempre ocupa o tamanho definido (bom para dados de tamanho constante, como UF ou sexo). **VARCHAR** é variável: ocupa só o necessário, mais 1 byte (bom para nome, e-mail).

LIGA COM ATRIBUTOS – UNSIGNED

O atributo `UNSIGNED` remove os negativos e dobra o limite positivo. Ex.: `SMALLINT` vai de -32.768 a 32.767, mas `UNSIGNED` vai de 0 a 65.535. Use quando o valor nunca é negativo (idade, quantidade).

NA PRÁTICA

Escolher o tipo certo **economiza espaço** e **evita erros** (guardar data como texto atrapalha ordenação e cálculo). **Próxima ação:** ao criar uma tabela, pergunte para cada coluna: número, texto ou data? E qual o menor tipo que comporta os valores?

18 Funções de string (texto)

estudado em 26 jun 2026

O QUE SÃO

Funções de **string** manipulam textos: padronizar, extrair partes, juntar e limpar dados. São rotina no trabalho com dados (normalizar nomes, separar um DDD, etc.).

Função	O que faz
CONCAT /	Une (concatena) textos. SQLite/Oracle: a b. MySQL/SQL Server: CONCAT(a,b).
LENGTH / LEN	Quantidade de caracteres.
UPPER / LOWER	Converte para maiúsculas / minúsculas.
SUBSTRING / SUBSTR	Extraí um pedaço (posição inicial + quantidade).
LEFT / RIGHT	N caracteres da esquerda / direita.
REPLACE	Substitui todas as ocorrências de um texto por outro.
LPAD / RPAD	Preenche à esquerda / direita até um tamanho.
LTRIM / RTRIM / TRIM	Remove espaços (ou caracteres) do início / fim / ambos.
SPLIT_PART	Divide por um delimitador e pega a parte na posição escolhida.

CÓDIGO SQL

```
-- Nome em maiúsculas e o tamanho:
SELECT UPPER(NOME) AS maiusculo, LENGTH(NOME) AS tamanho
FROM PESSOAS;

-- Juntando texto com || :
SELECT NOME || ' tem ' || IDADE || ' anos'
FROM PESSOAS WHERE ID=1;

-- Eliana tem 20 anos
```

NOMES VARIAM POR SGBD

CONCAT VS || , LEN VS LENGTH , SUBSTRING VS SUBSTR . O conceito é o mesmo; confira a documentação do seu banco. No app (SQLite) use || e LENGTH .

NA PRÁTICA

Limpeza e padronização de texto são constantes em Dados. **Próxima ação:** no modo Praticar, teste UPPER , LENGTH e || na tabela PESSOAS.

19 Funções numéricas e de data

estudado em 26 jun 2026

FUNÇÕES NUMÉRICAS

Realizam cálculos e ajustes em valores numéricos.

Função	O que faz
POWER / POW	Potência. <code>POWER(2,3) = 8</code> .
ROUND	Arredonda para o mais próximo. <code>ROUND(28.456,1) = 28.5</code> .
SQRT	Raiz quadrada. <code>SQRT(9) = 3</code> .
TRUNC	Corta os decimais sem arredondar. <code>TRUNC(111.15) = 111</code> .
ABS	Valor absoluto (sem sinal). <code>ABS(-7) = 7</code> .
MOD / %	Resto da divisão. <code>10 % 3 = 1</code> .

CÓDIGO SQL

```
SELECT ROUND(28.456, 1), SQRT(9), ABS(-7);  
-- 28.5 | 3 | 7
```

ROUND VS TRUNC

ROUND arredonda (28.6 vira 29). **TRUNC** corta (28.6 vira 28). Use **ROUND** para valores finais; **TRUNC** quando quer só a parte inteira.

FUNÇÕES DE DATA

Manipulam datas e horas: extrair partes, somar intervalos, calcular diferenças.

Função	O que faz
CURRENT_DATE	Data atual.
DATEADD	Soma um intervalo a uma data (dias, meses, anos).
DATEDIFF	Diferença entre duas datas.
DATE_PART / EXTRACT	Extraí uma parte (dia, mês, ano) de uma data.
DATE_TRUNC	Zera as partes menores (ex.: trunca por mês).
TO_TIMESTAMP	Converte texto em data/hora.

DATAS NO APP (SQLITE)

No SQLite (e no modo Praticar) datas se manipulam com `date()` e `strftime()`. Ex.: `strftime('%Y', data)` extrai o ano. `DATEADD/DATEDIFF` são do SQL Server; `EXTRACT/DATE_PART` são do PostgreSQL.

NA PRÁTICA

Funções de data são essenciais para relatórios por período (vendas por mês, idade da conta). **Próxima ação:** teste `ROUND` e `strftime('%Y', ...)` no Praticar.

20 Conversão e condicionais (CASE, COALESCE)

estudado em 26 jun 2026

CONVERSÃO DE TIPOS

Transformam um dado de um tipo para outro (texto em número, número em texto, etc.).

Função	O que faz
CAST / CONVERT	Converte tipos compatíveis. CAST('10' AS INTEGER).
TO_CHAR	Número ou data em texto formatado.
TO_DATE	Texto em data.
TO_NUMBER	Texto em número decimal.

CASE – DECISÃO DENTRO DO SQL

Cria lógica condicional na consulta. Sintaxe: CASE WHEN condição THEN resultado [ELSE padrão] END . Economiza código ao evitar tratar condições fora do banco.

CÓDIGO SQL

```
SELECT NOME,
CASE
  WHEN IDADE >= 40 THEN 'veterano'
  WHEN IDADE >= 30 THEN 'adulto'
  ELSE 'jovem'
END AS faixa
FROM PESSOAS;
```

IF / IIF

IF(condição, valor_se_verdadeiro, valor_se_falso) faz o mesmo que um CASE simples. No SQLite chama-se IIF. Ex.: IIF(IDADE >= 18, 'maior', 'menor') .

COALESCE – PRIMEIRO VALOR NÃO-NULO

Percorre uma lista e retorna o **primeiro valor que não for NULL**. Analogia: ligar para o cliente: tenta o comercial; se vazio, o celular; se vazio, o fixo. COALESCE(comercial, celular, fixo) .

NULLIF – NULL SE FOREM IGUAIS

Compara dois valores: se forem **iguais**, retorna **NULL**; se diferentes, retorna o primeiro. Útil para evitar divisão por zero (NULLIF(divisor, 0)) ou destacar diferenças (meta x realizado).

CÓDIGO SQL

```
SELECT COALESCE(NULL, NULL, 'fixo'); -- fixo
SELECT NULLIF(50, 50); -- NULL (iguais)
```

NA PRÁTICA

`CASE` classifica; `COALESCE` garante um valor mesmo com campos vazios; `NULLIF` protege cálculos. São o que deixa a consulta "inteligente" e segura contra nulos.

21 JOINS — juntando tabelas

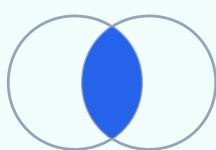
estudado em 26 jun 2026

O QUE É UM JOIN

JOIN (juntar) combina linhas de **duas ou mais tabelas** com base em colunas em comum (geralmente PK de uma e FK de outra). É o que permite ver, num só resultado, dados espalhados em tabelas diferentes.

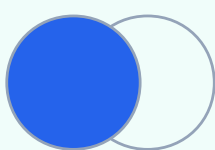
Atenção: em grandes volumes, JOINS têm **alto custo computacional** e podem deixar a consulta lenta.

Os quatro tipos (diagrama de Venn)



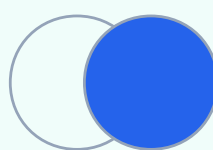
INNER JOIN

só o que casa nas duas



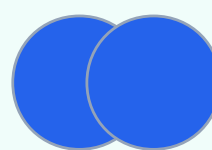
LEFT JOIN

tudo da esquerda



RIGHT JOIN

tudo da direita



FULL JOIN

tudo dos dois lados

Tipo	Retorna
INNER JOIN	Só as linhas que casam nas duas tabelas (a interseção).
LEFT JOIN	Todas as linhas da tabela da esquerda; o que não casa na direita vem NULL.
RIGHT JOIN	Todas as linhas da tabela da direita; o que não casa na esquerda vem NULL.
FULL OUTER JOIN	Todas as linhas dos dois lados; preenche com NULL onde não há par.

COMO LER UM JOIN

`FROM A JOIN B ON A.chave = B.chave`. O `ON` diz por qual coluna as tabelas se ligam. Use apelidos (A, B) para encurtar.

CÓDIGO SQL

```
-- Pessoa e produto de cada compra (INNER JOIN):  
SELECT P.NOME, PR.NOME  
FROM COMPRAS C  
JOIN PESSOAS P ON P.ID = C.ID_PESSOA  
JOIN PRODUTOS PR ON PR.ID = C.ID_PRODUTO;  
  
-- Todas as pessoas, mesmo quem não comprou (LEFT JOIN):  
SELECT P.NOME, C.ID_PRODUTO  
FROM PESSOAS P  
LEFT JOIN COMPRAS C ON C.ID_PESSOA = P.ID;
```

ACHAR O QUE NÃO TEM PAR

Um `LEFT JOIN` seguido de `WHERE coluna_da_direita IS NULL` traz só o que existe **apenas na esquerda**. Ex.: produtos que nunca foram comprados.

NA PRÁTICA

INNER = interseção; **LEFT/RIGHT** = um lado inteiro + o que casa; **FULL** = tudo. Para relatórios reais (cliente + pedidos + produtos) o `JOIN` é diário.

22 UNION – empilhando resultados

estudado em 26 jun 2026

O QUE É UNION

UNION combina (empilha) as linhas de **duas ou mais consultas** num único resultado. Diferença para o JOIN: o JOIN junta **colunas** (lado a lado); o UNION junta **linhas** (uma embaixo da outra).

ANALOGIA

Você tem uma lista de compras do mercado e outra da farmácia. O UNION é escrever todos os itens num papel só, um abaixo do outro.

DUAS REGRAS OBRIGATÓRIAS

1. **Mesmo número de colunas** nas duas consultas.
2. **Tipos e ordem compatíveis**: a coluna na mesma posição deve ter o mesmo tipo (texto com texto, número com número).

Comando	O que faz
UNION ALL	Junta tudo sem conferir. Mantém duplicatas. Mais rápido.
UNION	Junta e remove as duplicatas. Resultado limpo, mas exige mais esforço do banco.

CÓDIGO SQL

```
-- Empilha todos os IDs (com repetições):
SELECT ID_PESSOA FROM COMPRAS
UNION ALL
SELECT ID FROM PESSOAS;

-- Só os valores únicos (sem repetir):
SELECT ID_PESSOA FROM COMPRAS
UNION
SELECT ID FROM PESSOAS;
```

EXEMPLO VISUAL

Lista 1: Verde, Amarelo, Vermelho. Lista 2: Vermelho, Azul, Amarelo.

UNION ALL = 6 itens (Verde, Amarelo, Vermelho, Vermelho, Azul, Amarelo).

UNION = 4 itens únicos (Verde, Amarelo, Vermelho, Azul).

NA PRÁTICA

Use **UNION ALL** quando quiser tudo (e é mais rápido); use **UNION** quando precisa de uma lista sem repetições.

23 Subqueries (subconsultas)

estudado em 26 jun 2026

O QUE É

Uma **subquery** (subconsulta) é uma consulta **dentro** de outra. Como uma boneca russa: você faz um SELECT e, dentro dele, coloca outro SELECT.

DOIS USOS PRINCIPAIS

Como coluna: traz um valor calculado de outra tabela para cada linha.

Como filtro: limita os resultados com base numa lista gerada por outra consulta (WHERE ... IN (subconsulta)).

CÓDIGO SQL

```
-- Como filtro: só quem fez alguma compra
SELECT NOME FROM PESSOAS
WHERE ID IN (SELECT ID_PESSOA FROM COMPRAS);

-- Como coluna: total de compras de cada pessoa
SELECT NOME,
       (SELECT COUNT(*) FROM COMPRAS WHERE ID_PESSOA=P.ID) AS total
FROM PESSOAS P;
```

ATENÇÃO – DESEMPENHO

Não há limite para aninhar subconsultas, mas **muitas camadas deixam a consulta lenta** (o banco tem muito trabalho extra). Use com parcimônia.

NA PRÁTICA

Subconsultas respondem perguntas em duas etapas: "quem comprou?" e depois "traga os nomes desses". Ótimas para filtrar com base em outra tabela.

24 Views (visões)

estudado em 26 jun 2026

O QUE É UMA VIEW

Uma **view** (visão) é uma **tabela virtual**: não armazena dados próprios, é apenas um **SELECT salvo** com um nome. Você a consulta como se fosse uma tabela comum.

POR QUE USAR

Reuso: escreve a consulta difícil uma vez e salva.

Segurança: mostra só algumas colunas, escondendo dados sensíveis (ex.: salários).

Simplificação: transforma um código gigante em algo limpo de usar.

Característica	Detalhe
Dinâmica	Reflete os dados atuais: se a tabela muda, a view mostra a mudança.
Permanente	Continua existindo mesmo após reiniciar o servidor.
Leve	Não ocupa espaço extra: é só a instrução salva.

CÓDIGO SQL

```
-- Criar:
CREATE VIEW vw_adultos AS
  SELECT NOME, IDADE FROM PESSOAS WHERE IDADE >= 18;

-- Consultar como se fosse tabela:
SELECT * FROM vw_adultos;
```

COMANDOS

CREATE VIEW nome AS [SELECT] cria; ALTER VIEW altera a consulta; DROP VIEW apaga a view. **DROP VIEW não apaga os dados** das tabelas originais, só a visão.

NA PRÁTICA

Views deixam consultas complexas reutilizáveis e seguras (ótimo para esconder colunas sensíveis e padronizar relatórios).

FIM DA MATÉRIA DE SQL

Este caderno cobre o SQL do básico ao intermediário: dos fundamentos às consultas avançadas, funções, JOINS, UNION, subqueries e views. As próximas matérias (Visualização de Dados, Python, Power BI, IA) entram no mesmo app, cada uma como seu próprio caderno.